034IN -FONDAMENTI DI AUTOMATICA - FUNDAMENTALS OF AUTOMATIC CONTROL

FIRST PARTIAL EXAMINATION - A.Y. 2023/2024

May 3, 2024

Problem 1 - Realisation in Control Form for a SISO Not Strictly Proper System

Tag List: `[STANDARD ASSESSMENT] [PARTIAL CREDIT] [CONTROL TOOLBOX]`

Realisation in Control Form: a Recap

Consider the generic **not strictly proper** transfer function

$$G(s) = \frac{\beta_n s^n + \beta_{n-1} s^{n-1} + \cdots + \beta_1 s + \beta_0}{s^n + \alpha_{n-1} s^{n-1} + \cdots \alpha_1 s + \alpha_0} = \beta_n + \frac{\gamma_{n-1}\, s^{n-1} + \gamma_{n-2}\, s^{n-2} + \cdots \gamma_1\, s + \gamma_0}{s^n + \alpha_{n-1} s^{n-1} + \cdots \alpha_1 s + \alpha_0}$$

Then the **realisation in Control Form** is

$$\begin{cases} \begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \\ \vdots \\ \dot{x}_{n-1}(t) \\ \dot{x}_n(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 & 0 \\ 0 & 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 0 & 1 \\ -\alpha_0 & -\alpha_1 & -\alpha_2 & \cdots & -\alpha_{n-2} & -\alpha_{n-1} \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_{n-1}(t) \\ x_n(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} u(t) \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad (\star) \\ y(t) = \begin{bmatrix} \gamma_0 & \gamma_1 & \cdots & \gamma_{n-2} & \gamma_{n-1} \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_{n-1}(t) \\ x_n(t) \end{bmatrix} + \begin{bmatrix} \beta_n \end{bmatrix} u(t) \end{cases}$$

Refer to **Part 4** of the course material for details.

Assignment

Consider the LTI system described by the transfer function

$$G(s) = \frac{2\, s^3 + s^2 - 12\, s + 5}{s^3 + 2s^2 + 4s + 3}$$

Determine the realisation in Control Form of such a system, according to Eq. $(\star)$.

Your code shall

- Define the transfer function $G(s)$ as an LTI system in MATLAB, using the function `tf( )`. For information and/or examples on the use of `tf( )`, please consult the MATLAB documentation.
- Retrieve the numerator and denominator coefficients of the transfer function $G(s)$. Use the function `tfdata()` [**mandatory**]. The coefficients must be stored in **numerical row vectors**: if help is needed regarding how to obtain the coefficients from a `tf` object, please consult the MATLAB documentation of tfdata( ). Save the coefficients in the row vectors named **`numG`** and **`denG`** respectively.
- **Polynomial Division [mandatory]**: Perform **polynomial division** of the numerator by the denominator, determining quotient and polynomial remainder. The quotient β_n will be the matrix D, while the polynomial remainder allows us to determine the matrices A, B and C expressed in equation $(\star)$. Save the results in the following MATLAB variables: store the quotient in the variable **`betaN`** and the remainder of the polynomial division in the row vector **`GgammaV`**.
- if help is needed regarding how to compute a polynomial division, please consult the MATLAB documentation of deconv( ).
- **Important**: The `deconv( )` function usually returns the coefficients of the remainder polynomial with the same number of divider elements. Indeed, the polynomials (divider and remainder) do not have the same degree. This means that the vector of coefficients of the remainder polynomial has null values (the coefficients associated with the highest-degree monomials) as its initial elements, up to the coefficient of the actual monomial of the highest degree of the remainder itself. **These null values must be eliminated before proceeding further.**
- **ToDo {mandatory]**: Copy the contents of **`GgammaV`** in a new row vector named **`gammaG`**. Check if the first element in **`gammaG`** is zero. In this case, provide a few lines of code to remove the heading zeros from **`gammaG`**. Do the same (store the contents of **`GgammaV`** in the row vector **`gammaG`**) even if you didn't perform any leading zeros removal task.
- **From now on**, your code must use the row vectors **`gammaG`** and **`denG`** as the containers of the polynomial coefficients at the numerator and denominator of the strictly proper part of $G(s)$.
- According to Eq. $(\star)$, assemble the numerical matrices **`A1`**, **`B1`**, and **`C1`** of the **realisation in Control Form**. Insert the coefficients stored in the vectors **`gammaG`** and **`denG`** into the proper positions in the matrices. Use **`betaN`** to create the matrix **`D1.`**
- Create the LTI system model **`CF1_sys`**, exploiting the matrices **`A1`**, **`B1, C1`** and **`D1`** and the function `ss()`.
- Compute the tf model of **`CF1_sys`**, as `tf(CF1_sys)`. Save the resulting LTI tf object in the MATLAB variable **`G_result.`** **Note**: it should be the same as **`Gs`**.

Solution Proposal

```
% ***** Assignment  *****

% the tf building helper
s = tf('s');

% assign the TF G(s)
Gs = (2*s^3+s^2-12*s+5)/(s^3+2*s^2+4*s+3)
```

```
Gs =

  2 s^3 + s^2 - 12 s + 5
  ----------------------
  s^3 + 2 s^2 + 4 s + 3

Continuous-time transfer function.
Model Properties
```

```
% retrieve the vectors of coefficients
[numG, denG] = tfdata(Gs, 'v')
```

```
numG = 1×4
     2     1   -12     5
denG = 1×4
     1     2     4     3
```

```
% compute the polynomial division
[betaN, VgammaG] = deconv(numG, denG)
```

```
betaN =
2
VgammaG = 1×4
     0    -3   -20    -1
```

```
gammaG = VgammaG; % assign numG

% removing the leading zeros in gammaG
while (gammaG(1)==0)
    gammaG(1) = [];
end



% order of the LTI system <--> degree of the polynomial at the denominator
n = numel(denG)-1;

% degree of the polynomial at the numerator
m = numel(gammaG)-1;

% ------- assembling the matrices --------
D1 = betaN;

B1 = [zeros(n-1, 1); +1];

C1 = [fliplr(gammaG), zeros(n-(m+1), 1)];

A1 = [zeros(n-1, 1), eye(n-1); ...
      -fliplr(denG(2:end))];

CF1_sys = ss(A1, B1, C1, D1)
```

```
CF1_sys =

  A =
       x1  x2  x3
   x1   0   1   0
   x2   0   0   1
   x3  -3  -4  -2

  B =
       u1
   x1   0
   x2   0
   x3   1

  C =
        x1   x2   x3
   y1   -1  -20   -3

  D =
       u1
   y1   2

Continuous-time state-space model.
Model Properties
```

```
% final check
```

```
G_result = tf(CF1_sys)
```

```
G_result =

  2 s^3 + s^2 - 12 s + 5
  ----------------------
  s^3 + 2 s^2 + 4 s + 3

Continuous-time transfer function.
Model Properties
```